

RÚBRICA ANALÍTICA DE EVALUACIÓN (100 PUNTOS TOTALES)

La evaluación se compone de una Base Fija Obligatoria (40 Puntos) común para ambas rutas, y un Menú de Pilares Cloud Electivos (60 Puntos) de los cuales el estudiante debe seleccionar e implementar exactamente dos (2) de los cuatro (4) disponibles.

PARTE I: BASE FIJA OBLIGATORIA (40 PUNTOS)

Criterio	Excelente (100%-90%)	Bueno (89%-75%)	En Desarrollo (74%-60%)	Pts
Aplicación Base Funcional (Saber Hacer)	La aplicación base (ABM creado en Ruta A o Proyecto Activo de Ruta B) es completamente funcional. El modelo de datos y la interfaz de usuario operan sin errores lógicos ni bugs visibles.	La aplicación es funcional en sus flujos principales, pero se detectan pequeños bugs cosméticos o fallas de validación de datos menores.	La aplicación presenta fallas de persistencia de datos (ej. caídas al guardar/actualizar) o la funcionalidad ABM está incompleta en alguno de sus flujos.	15 pts
Seguridad, Cifrado y Manejo de Secretos (Saber Conocer)	Los secretos (credenciales de base de datos, API keys, tokens de la nube) están 100% cifrados y desacoplados del código fuente. Se hace uso estricto de secretos de GitHub, Kubernetes Secrets o Parameter Store consumidos de forma segura en tiempo de ejecución.	Se aíslan los secretos de la aplicación en variables de entorno, pero se detectan configuraciones con permisos excesivos en la nube o dependencias desactualizadas con vulnerabilidades conocidas.	Existen malas prácticas menores en el manejo de secretos (ej. archivos .env subidos temporalmente al historial de Git, aunque corregidos posteriormente) o canales de comunicación sin cifrar.	10 pts
Documentación Técnica y Defensa Oral (Saber Conocer / Ser)	Documentación impecable con diagramas arquitectónicos correctos del proveedor, ADRs técnicamente justificadas y Runbook claro. Defensa oral sólida que demuestra un dominio individual e independiente absoluto de los conceptos de	Presenta la documentación completa, pero faltan precisiones técnicas en la guía de despliegue o la estimación de costos del consumo de nube es vaga.	Documentación desordenada o incompleta. Durante la defensa individual, el estudiante tiene dificultades conceptuales marcadas para explicar la interacción de los componentes desplegados.	15 pts

Criterio	Excelente (100%-90%)	Bueno (89%-75%)	En Desarrollo (74%-60%)	Pts
	computación en la nube.			

TOTAL BASE OBLIGATORIA: 40 puntos

PARTE II: PILARES CLOUD ELECTIVOS (60 PUNTOS)

Cada estudiante debe seleccionar e implementar dos (2) pilares de la siguiente lista según la naturaleza de su proyecto y ruta seleccionada. Cada pilar seleccionado tiene un valor de 30 puntos ($2 \times 30 = 60$ pts).

Al momento de la entrega, el estudiante deberá declarar formalmente cuáles pilares presentará ante el docente.

PILAR 1: Contenedores y Orquestación (Docker & Kubernetes)

Requisitos:

Dockerización optimizada de la aplicación utilizando técnicas multi-stage y desacoplamiento de la base de datos de la capa de cómputo. La base de datos debe desplegarse como servicio administrado en la nube (RDS, Azure SQL, Cloud SQL) o mediante StatefulSets con PersistentVolumeClaims si se conteneriza. Configuración de manifiestos del orquestador (Kubernetes / K3s / AKS / GKE) incluyendo Deployment, Services, ConfigMaps y Secrets, además de políticas de autoescalado (HPA) ante simulación de tráfico.

Contexto por Ruta:

- Ruta A (AWS): Se permite desplegar un clúster ligero (K3s o MicroK8s) sobre instancias EC2 utilizando el LabRole para evitar los altos costos y las trabas administrativas de EKS en el Learner Lab.
- Ruta B (Libre): Se evalúa el uso nativo de servicios administrados de Kubernetes (ej. Azure AKS o GCP GKE) o motores de contenedores equivalentes acordes a su arquitectura.

Nivel de Logro	Descripción del Desempeño	Puntos
Excelente	Imágenes Docker optimizadas (multi-stage builds), uso correcto de Secrets/ConfigMaps en K8s, base de datos desacoplada mediante servicio administrado o PersistentVolumes, despliegue con HPA activo y autoescalado comprobado bajo simulación de tráfico.	30 pts
Bueno	Contenedores funcionales en el orquestador con manifiestos correctos, pero las variables sensibles están expuestas en texto plano dentro de los archivos YAML, o carece de HPA configurado.	24 pts
En Desarrollo	Aplicación contenerizada corriendo en el orquestador pero de forma estática (sin mapeo dinámico de servicios, fallas de persistencia de datos o mapeo de puertos incorrecto).	18 pts
Insuficiente	Fallas críticas en las imágenes de Docker (errores continuos de CrashLoopBackOff) o nula configuración del orquestador de contenedores.	9 pts

PILAR 2: Infraestructura como Código (IaC con Terraform)

Requisitos:

Aprovisionamiento de la infraestructura de red (VPC, Subredes, Firewalls / Security Groups) y recursos de cómputo/almacenamiento mediante Terraform de manera declarativa. Es obligatorio el uso de módulos

reutilizables y el manejo seguro del estado remoto de Terraform (backend en S3/Azure Blob/GCS con bloqueo dinámico cuando sea posible).

Contexto por Ruta:

- Ruta A (AWS Learner Lab): El código de Terraform debe configurarse para funcionar con credenciales temporales (aws_session_token) y heredar de manera explícita el ARN de LabRole para la creación de recursos. Para el backend remoto, se permite S3 sin DynamoDB lock debido a restricciones del Learner Lab.
- Ruta B (Libre): El código de Terraform debe estar adaptado al proveedor elegido (Azure, GCP, etc.) siguiendo las mejores prácticas de estructura de directorios y modularización nativa.

Nivel de Logro	Descripción del Desempeño	Puntos
Excelente	Infraestructura aprovisionada mediante Terraform de forma automatizada, estructurado en módulos reutilizables, parametrizado y con backend remoto funcional (S3/Azure Blob/GCS).	30 pts
Bueno	Código Terraform funcional pero plano (sin modularización, archivo monolítico) o con persistencia de estado exclusivamente local en el equipo de desarrollo.	24 pts
En Desarrollo	Terraform compila y despliega recursos sueltos (como servidores virtuales) pero carece de la configuración declarativa de la topología de red completa.	18 pts
Insuficiente	Uso nulo o meramente instrumental de Terraform (la gran mayoría de los recursos fueron levantados manualmente desde la consola gráfica del proveedor).	9 pts

PILAR 3: Automatización de Pipelines (CI/CD)

Requisitos:

Automatización del ciclo de vida del software mediante un pipeline (GitHub Actions, GitLab CI u homólogo). El flujo de ejecución debe activarse por triggers de Git, realizar análisis/pruebas unitarias (mínimo 3 pruebas que validen las operaciones CRUD principales), construir y escanear la seguridad de la imagen de Docker, subirla a un registro privado/público y actualizar el entorno de despliegue en la nube de forma automatizada.

Contexto por Ruta:

- Ruta A (AWS): El pipeline debe estar parametrizado para recibir e inyectar dinámicamente el AWS_SESSION_TOKEN de AWS Academy como secreto de repositorio para poder autenticar el despliegue automático. Se acepta despliegue semi-automático (build/test/scan automático + deploy manual trigger) debido a la expiración de credenciales cada 4 horas.
- Ruta B (Libre): Se permite el uso de herramientas nativas del proveedor (Azure DevOps, GCP Cloud Build) o herramientas universales parametrizadas para su respectivo flujo de autenticación.

Nivel de Logro	Descripción del Desempeño	Puntos
Excelente	Pipeline completamente integrado de extremo a extremo (Build, Test con mínimo 3 pruebas unitarias, Security Scan, Push y Deploy automático/semi-automático en la nube) con un manejo impecable de secretos.	30 pts

Nivel de Logro	Descripción del Desempeño	Puntos
Bueno	Pipeline funcional que ejecuta build, test y push de la imagen Docker de forma correcta, pero la actualización final en el entorno de despliegue requiere pasos manuales significativos del estudiante.	24 pts
En Desarrollo	El pipeline de automatización se limita a ejecutar pruebas de linter o compilación de software básica, sin conexión o despliegue interactivo con la infraestructura cloud.	18 pts
Insuficiente	Inexistencia del pipeline de CI/CD. El despliegue de las actualizaciones de software se realiza de manera completamente manual dentro del servidor.	9 pts

PILAR 4: Modernización de Servicios y Arquitectura Serverless

Requisitos:

Desacoplamiento de tareas asíncronas pesadas (ej. procesamiento de archivos, generación de reportes PDF, envío de notificaciones automáticas o compresión de imágenes) desde la aplicación principal hacia una arquitectura reactiva basada en eventos utilizando almacenamiento de objetos y funciones serverless (FaaS: AWS Lambda, Azure Functions, Cloud Functions).

Ejemplos válidos de arquitecturas:

- S3 + Lambda (event-driven file processing)
- API Gateway + Lambda (REST API serverless)
- EventBridge + Lambda (event-driven workflows)
- SQS/SNS + Lambda (message-driven async processing)

Contexto por Ruta:

- Ruta A (AWS): Configuración de AWS Lambda asociada a un trigger de S3, EventBridge, SQS o API Gateway, utilizando de manera obligatoria el ARN de LabRole como rol de ejecución de la función.
- Ruta B (Libre): Implementación de arquitecturas sin servidor o basadas en eventos (FaaS / BaaS) compatibles con el proveedor seleccionado y que den soporte lógico al proyecto del estudiante.

Nivel de Logro	Descripción del Desempeño	Puntos
Excelente	Integración asíncrona perfecta y reactiva desencadenada por eventos de almacenamiento o mensajes (ej. S3 Trigger → Lambda, SQS → Lambda) comunicada nativamente con la aplicación base.	30 pts
Bueno	Se implementa la función serverless en la nube, pero esta es invocada de forma síncrona mediante peticiones HTTP directas (sin reaccionar a eventos nativos del proveedor).	24 pts
En Desarrollo	La función serverless está desplegada en la nube, pero carece de lógica de integración real con el flujo de datos de la aplicación base o del proyecto del estudiante.	18 pts
Insuficiente	Toda la lógica de procesamiento sigue acoplada dentro del servidor tradicional sin evidencia de evolución hacia arquitecturas Serverless.	9 pts